

Sleak is a simple tool that monitors SWT *graphics* resources. (Applications typically don't leak *widget* resources because of rule 2). You can use *Sleak* to detect leaks in SWT application code.

Here is the source code: [Sleak.java](#).

To use the tool, put the *Sleak* class on your class path and then find the place in your application code where you create the *Display*:

```
Display display = new Display();
```

Before creating the display, create a *DeviceData* and set the **data.tracking** flag to true.

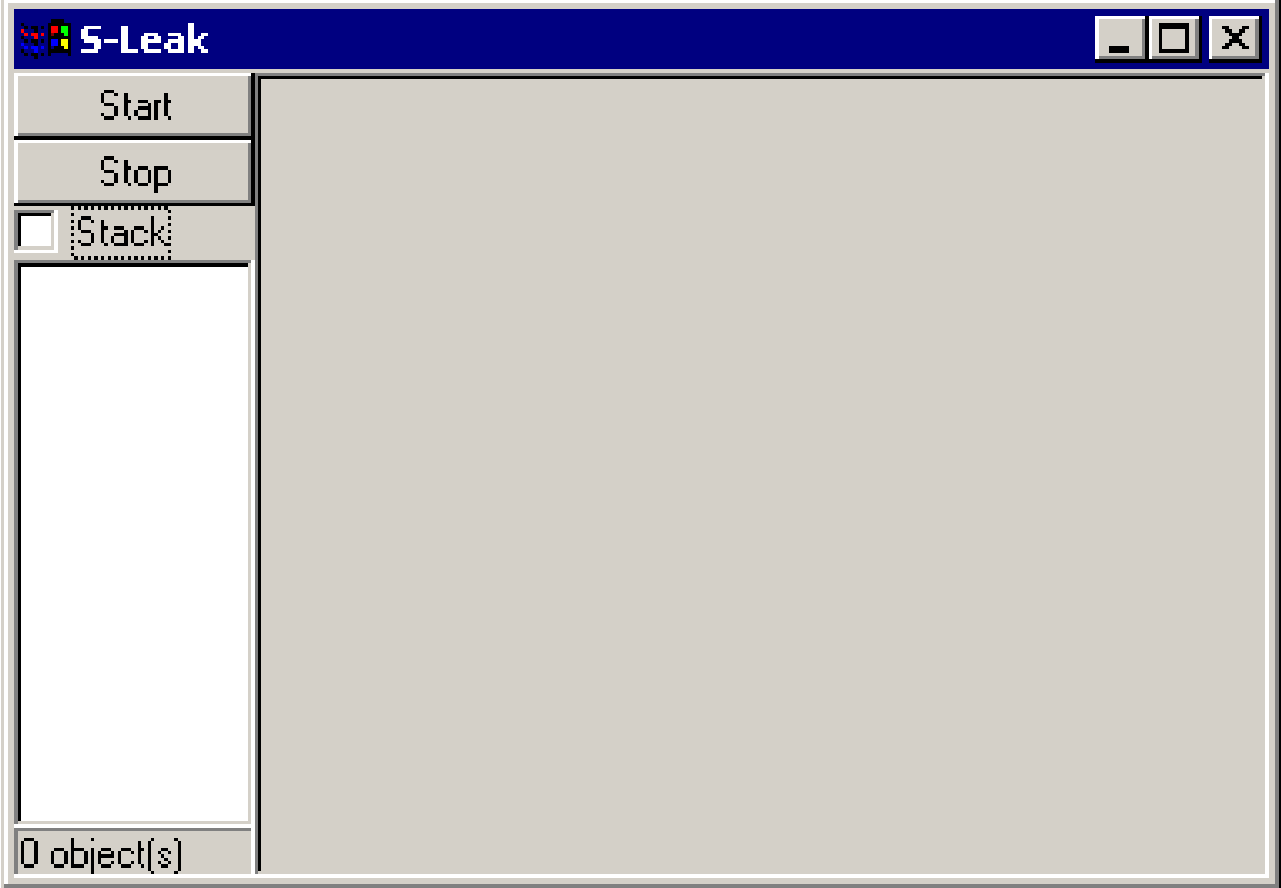
Then create the *Display* with this new *DeviceData* object.

Then create an instance of *Sleak* and open it.

Your new code will look something like this:

```
DeviceData data = new DeviceData();
data.tracking = true;
Display display = new Display(data);
Sleak sleak = new Sleak();
sleak.open();
```

Now run your code as usual. When the *Sleak* window comes up, it looks like this:



When you press the "Start" button, *Sleak* collects and displays a list of the SWT *graphics* resources that are currently allocated. At this point, perform the action in your application that you suspect is leaking. When you press the "Stop" button, *Sleak* displays the list of resources that were allocated since you pressed "Start". Select a resource in the list on the left and it will be drawn in the canvas on the right. Select the "Stack" checkbox to see a stack trace showing where the resource was created. This does not necessarily imply that this resource was leaked - just that it currently exists. If you believe that this particular resource is no longer needed and should have been disposed, then you have discovered a leak.

Example

Look at [LeakExample.java](#). You suspect that this code is leaking, and decide to use *Sleak* to verify your theory, and to investigate a possible fix.

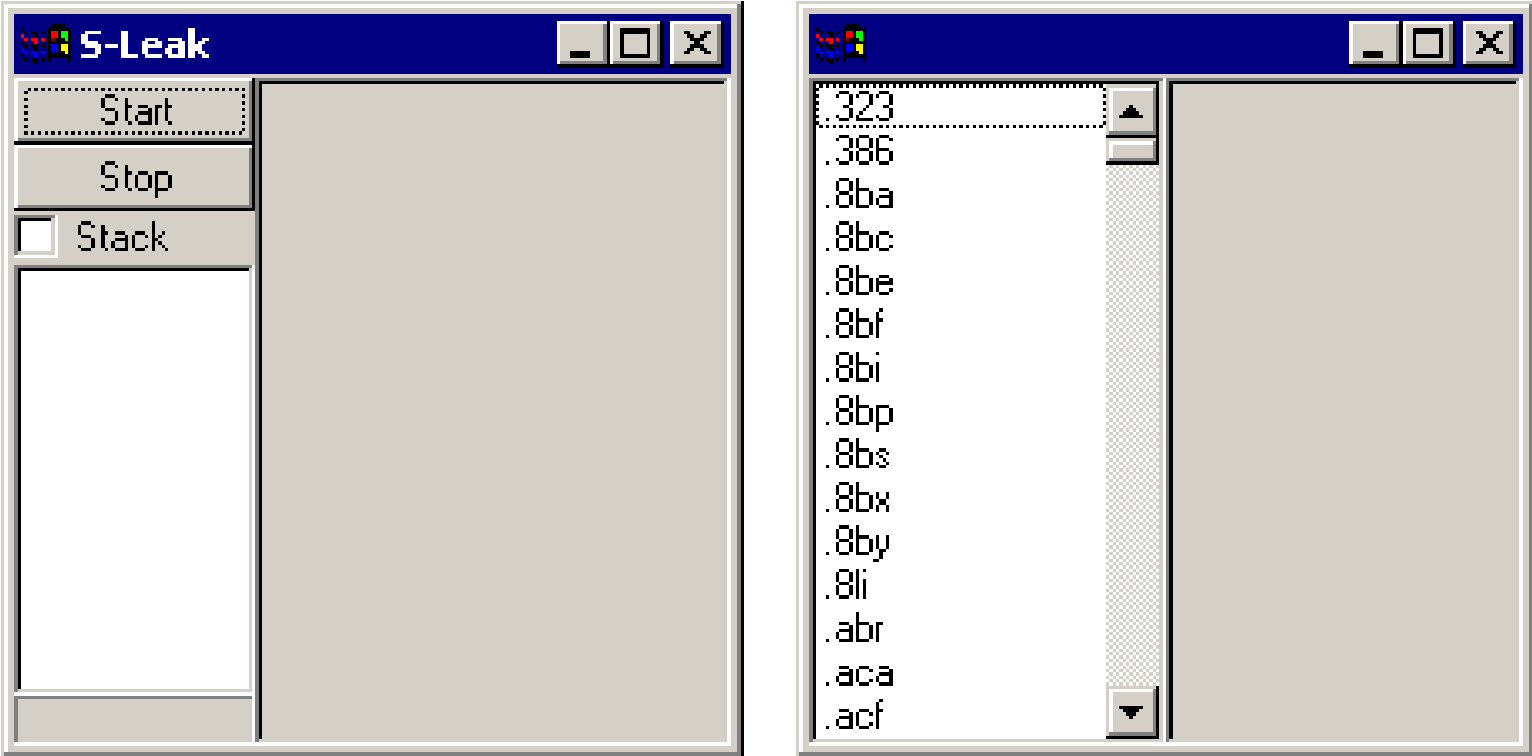
Find the line of code that creates the *Display*:

```
display = new Display();
```

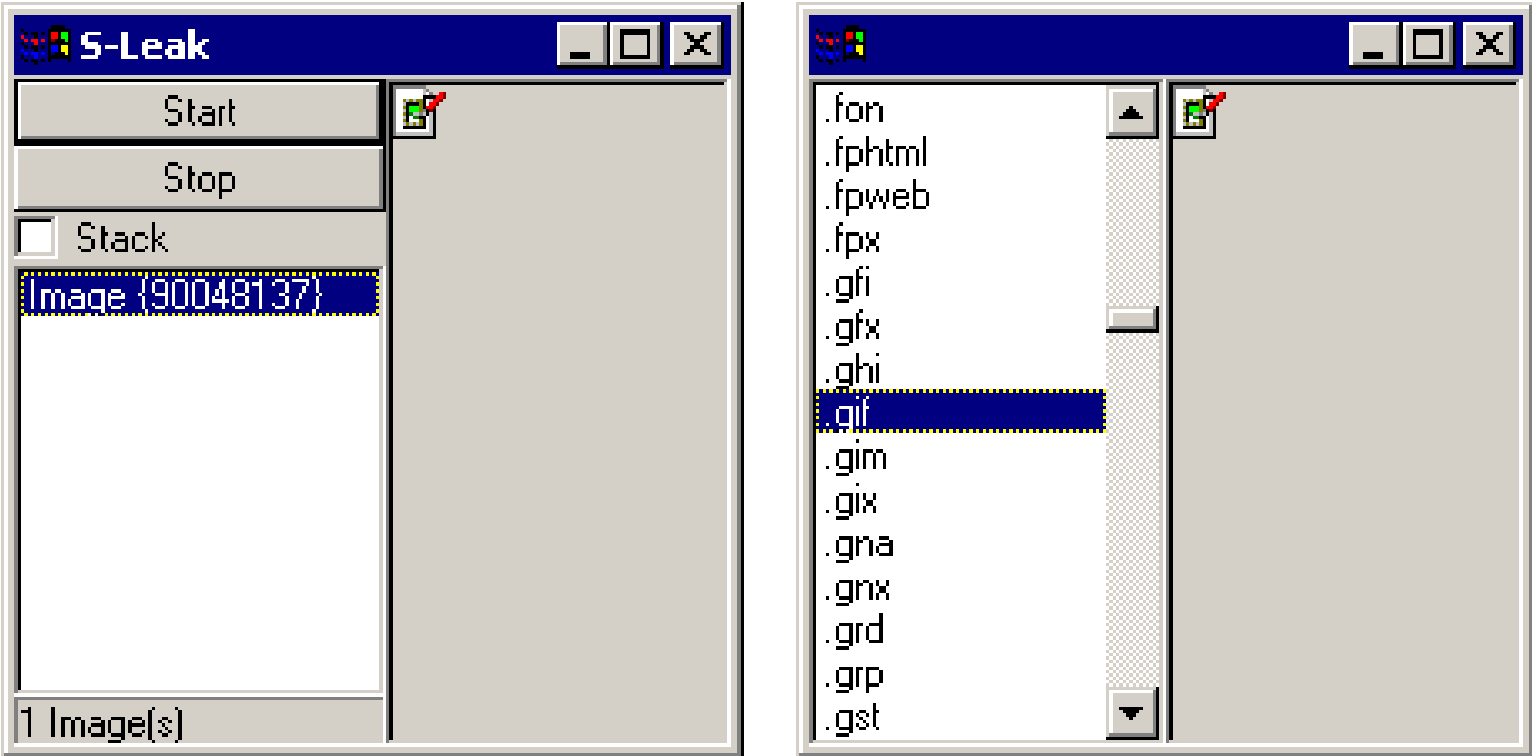
And rewrite the beginning of the **main()** method as follows:

```
public static void main(String[] args) {
    DeviceData data = new DeviceData();
    data.tracking = true;
    display = new Display(data);
    Sleak sleak = new Sleak();
    sleak.open();
    ...
}
```

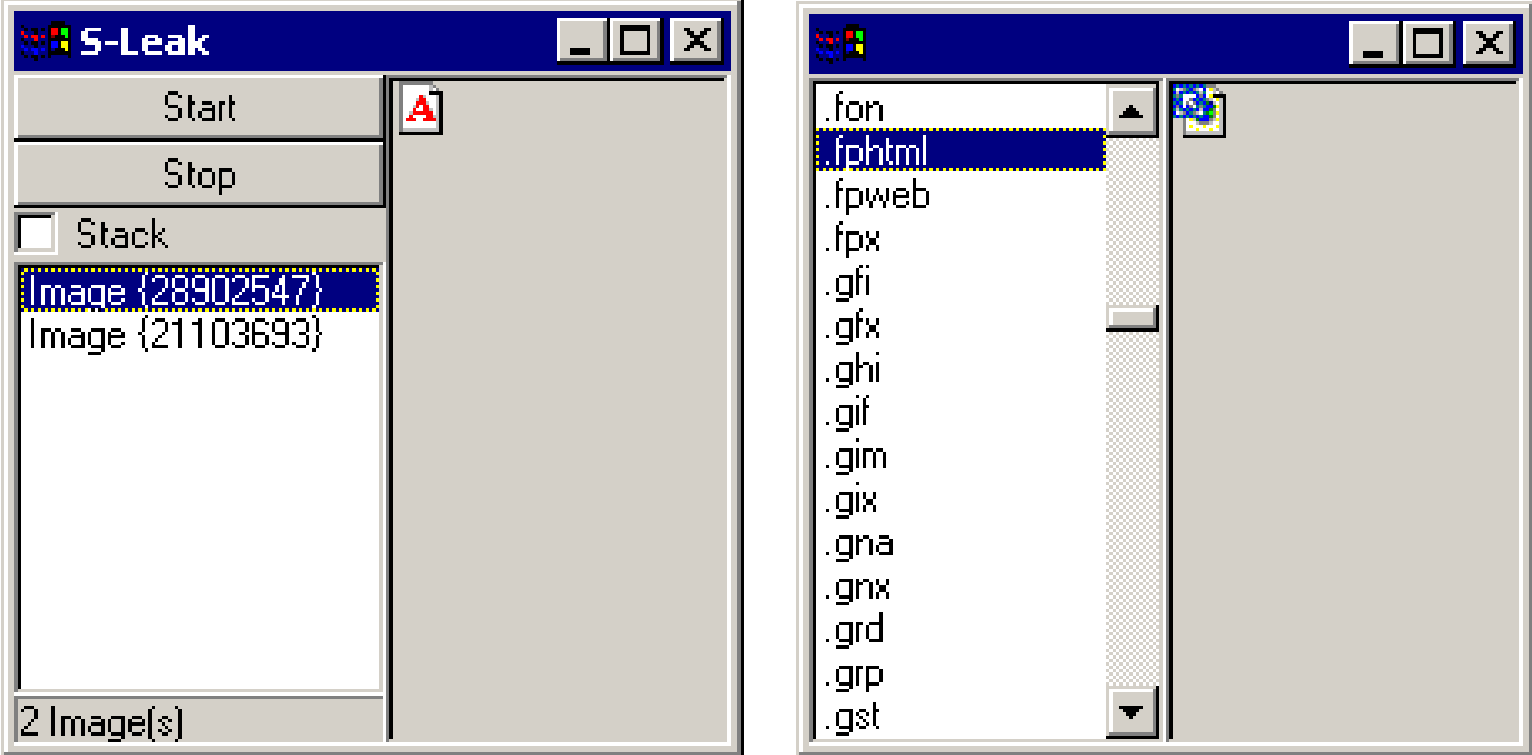
Run the *LeakExample* class as usual, and two windows open: your application, and the *Sleak* window. Press *Sleak's* "Start" button, and notice that no resources have yet been allocated - nothing shows up in *Sleak's* resource list.



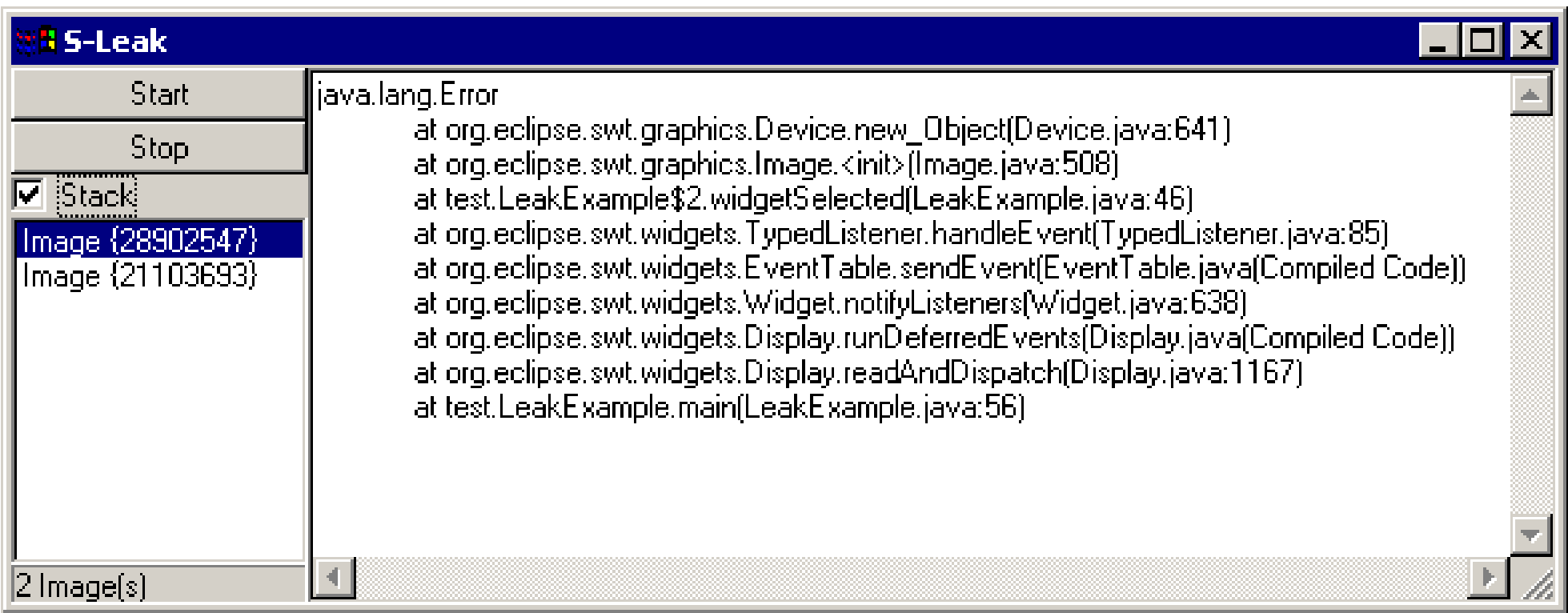
Return to the *LeakExample* window and click on an item in the list that has an associated image, say, '.gif'. Press "Start" again, and now you can see that one image has been created. Select the image and see that it is the one currently being displayed in the example. This will be your "starting point". No resources have been leaked yet because there is only one image, and it is still being used.



Now click on some other items in the example list, and press "Stop". *Sleak* shows 2 images that were both created since "Start" was pressed, but only one is being displayed in the application.



Select one of the new images, and check "Stack" to see where the image was created:



The image was created in the **widgetSelected** method in the *LeakExample* class, so this is the method to look at to see if the example is leaking. Here is the code for the method:

```
public void widgetSelected(SelectionEvent e) {
    image = null;
    String[] selection = list.getSelection();
    if (selection.length != 0) {
        Program program = Program.findProgram(selection[0]);
        if (program != null) {
            ImageData imageData = program.getImageData();
            if (imageData != null) {
                if (image != null) image.dispose();
                image = new Image(display, imageData);
            }
        }
    }
    canvas.redraw();
}
```

The **image = null** at the beginning of the method looks suspicious. This was added so that the paint did not draw the old image if an item with no image was selected. Before setting **image** to null, the image should be disposed, as follows:

```
if (image != null) {
    image.dispose();
    image = null;
}
```

After adding this dispose code, notice that you can delete the old line that disposed the image right before creating a new one:

```
if (image != null) image.dispose();
```

Now the code has been cleaned up, so run it under *Sleak* again and verify that nothing is leaking. One last thing for completeness and to get into good habits: you should also check if **image** needs disposing after the read and dispatch loop, or in a dispose listener on the shell.